🔒 Propuesta Arquitectural — Sistema de Permisos Telegram v2.0

Proyecto: Intrale Platform | **Fecha:** 2026-03-02 | **Version:** 2.0 | **Autores:** Guru + UX + PO + Arquitecto

Contenido

1. Diagnóstico: por qué fallan las notificaciones
2. Arquitectura propuesta
3. Diseño UX de mensajes
4. Requisitos funcionales y no funcionales
5. Gaps críticos a resolver
6. Plan de implementación
7. Métricas de éxito

1. Diagnóstico: por qué fallan las notificaciones

Investigación de Guru reveló **3 causas raíz documentadas** en el repo oficial de Claude Code:

1.1 — PermissionRequest se dispara de más (Issue #29212)

El hook `PermissionRequest` se ejecuta para **toda evaluación de permisos**, incluyendo las que Claude ya sabe que va a auto-aprobar desde settings. Esto genera 392 invocaciones donde solo 121 requerían acción del usuario.

1.2 — Race condition entre hook y diálogo (Issue #12176)

Si el hook tarda más de ~1.5 segundos en responder, Claude Code muestra el prompt local **antes** de recibir la respuesta del hook. El usuario ve el prompt en consola aunque el hook ya envió el mensaje a Telegram.

> **Bug confirmado:** Claude Code captura una snapshot de los hooks al iniciar sesión. Si un hook falla repetidamente, puede degradarse internamente. Las modificaciones a hooks mid-session NO se recargan.

1.3 — Notification(permission_prompt) solo se dispara cuando el diálogo SE MUESTRA

Contrario a lo que asumimos, `Notification(permission_prompt)` se dispara **solo cuando el usuario ve el diálogo en consola**, NO cuando el hook lo resuelve programáticamente. Esto significa que cuando el approver auto-aprueba rápido (vía hook), la Notification NO se dispara en absoluto.

Evento	Cuándo se dispara	Puede controlar decisión
`PermissionRequest`	En toda evaluación de permisos (incl. auto-aprobados)	Sí: allow/deny
`Notification(permission_prompt)`	Solo cuando el diálogo se muestra al usuario en consola	No (solo side-effects)

> **Implicación:** Usar `Notification` como fallback es correcto — solo se dispara cuando el PermissionRequest falló o no corrió, que es exactamente cuando necesitamos alertar.

1.4 — Bug de process.kill(pid, 0) en Windows

La verificación de PID con `process.kill(pid, 0)` puede dar **falsos negativos** en Windows. Cuando hay diferencia de permisos entre procesos, lanza `EPERM` (el proceso EXISTE pero no tenemos permiso para señalizarlo). El código actual trata `EPERM` como "proceso muerto", lo cual es incorrecto.

```
 // ACTUAL (bug):
try { process.kill(pid, 0); return true; }
catch(e) { return false; }  // EPERM = false ❌

// CORRECTO:
try { process.kill(pid, 0); return true; }
catch(e) { return e.code === "EPERM"; }  // EPERM = true ✅
```

2. Arquitectura propuesta

2.1 — Principio de diseño: dos canales independientes

En lugar de intentar coordinar hooks frágiles con archivos PID y markers, la arquitectura v2 usa **dos canales completamente independientes** que convergen en el Commander:

Canal	Trigger	Función	Fiabilidad
Canal A: Approver	`PermissionRequest` hook	Envía mensaje con botones, espera respuesta, controla la decisión	~70% (el hook no siempre se dispara)
Canal B: Watchdog	`Notification(permission_prompt)` + health-check periódico	Detecta prompts huérfanos y notifica al usuario	~95% (Notification + health-check como doble red)

2.2 — Flujo completo con fallbacks

```
Claude Code necesita permiso
    |
    ├── PermissionRequest hook se dispara?
    |   |
    |   ├─ SÍ → permission-approver.js
    |   |       |
    |   |       ├─ Auto-approve? → allow + marker → FIN
    |   |       |
    |   |       ├─ Enviar Telegram OK? → esperar botones → FIN
    |   |       |
    |   |       └─ Telegram falla? → RETRY 1x → si falla → exit(0)
    |   |                                                  |
    |   └─ NO ──────────────────────────────────────┬──┐
    |                                                  |   |
    |                                       Claude muestra prompt local
    |                                                  |
    |                                       Notification(permission_prompt) se dispara
    |                                                  |
    |                                       Canal B: Watchdog
    |                                            |
    |                                            ├─ Approver activo? → ignorar
    |                                            |
    |                                            └─ Sin approver → NOTIFICAR
    |                                                "🚫 Permiso pendiente en consola"
    |
    └── Health-check (cada 10 min): verifica prompts huérfanos
        Si detecta approver muerto con pregunta pending → NOTIFICAR
```

2.3 — Cambio clave: PID file ANTES del sendMessage

> **Gap crítico actual:** El approver escribe el PID file DESPUÉS de enviar el mensaje (línea 426). Hay una ventana de ~1-3 segundos donde la Notification puede dispararse, no encontrar PID file, y enviar un duplicado.

Fix: Mover la escritura del PID file al inicio del `processInput()`, antes de cualquier lógica. Si es auto-approve, se borra al salir. Si es Telegram, permanece durante el polling.

2.4 — Endpoint HTTP en el Commander para respuestas instantáneas

Reemplazar el polling de `pending-questions.json` (150ms) con un endpoint HTTP local en el Commander:

```
// En telegram-commander.js (ya tiene servidor HTTP en puerto 3100):
// Agregar endpoint:
//   GET /api/decision/:requestId  → long-poll hasta que haya respuesta
//   POST /api/decision/:requestId → registrar decisión (desde callback)

// En permission-approver.js:
// Reemplazar polling de archivo con:
//   fetch("http://localhost:3100/api/decision/" + requestId)
//   → respuesta instantánea cuando el usuario presiona botón
```

Ventajas: elimina 400 reads/segundo del filesystem, latencia < 5ms, sin race conditions de archivo.

3. Diseño UX de mensajes

3.1 — Jerarquía visual: cada estado es distinguible en 2 segundos

Estado 1: Permiso nuevo (requiere acción)

```
🔒 PERMISO REQUERIDO
──────────────────────────────────────
🤖 AndroidDev  ·  🔀 agent/1115 (#1115)
📌 Ejecutar tests E2E

🔧 Ejecutar emulador
$ ./qa/scripts/qa-android.sh --avd virtual…

⏳ Expira en 30 min  ·  1/4

[ ✅ Permitir ]  [ ✅✅ Siempre ]  [ ❌ Denegar ]
```

Cambios vs actual:

- Header distinguido `🔒 PERMISO REQUERIDO` (antes: `⚠️ Claude Code — Permiso requerido`)
- **3 botones** en vez de 2: se agrega `✅✅ Siempre` como botón (antes era solo texto libre)
- Cronómetro visible con posición en secuencia `1/4`

Estado 2: Recordatorio (usuario no respondió)

```
🔔 RECORDATORIO — permiso pendiente (2/4)
──────────────────────────────────────
🤖 AndroidDev  ·  📌 Ejecutar tests E2E

··· mismo permiso de hace 30 min ···
$ ./qa/scripts/qa-android.sh --avd virtual…

⏳ Expira en 15 min
[ ✅ Permitir ]  [ ✅✅ Siempre ]  [ ❌ Denegar ]
```

Estado 3: Aprobado / Rechazado (resolución)

```
✅ Permitido  ·  hace 2 min
──────────────────────────────────────
🤖 AndroidDev  ·  📌 Ejecutar tests E2E
$ ./qa/scripts/qa-android.sh --avd virtual…
```

```
↳ patrón guardado para futuras ejecuciones
```

Estado 4: Fallback (hook no corrió, prompt en consola)

```
📵 SIN CONEXIÓN — permiso en consola
────────────────────────────────────
El bot no pudo enviarte la notificación.
El agente está esperando en la PC.

Acción: ir a la consola y responder el prompt.
```

Estado 5: Expirado (timeout agotado)

```
⏱ Expirado — 4/4 intentos sin respuesta
────────────────────────────────────
El agente continuó con prompt local.
$ ./qa/scripts/qa-android.sh --avd virtual…
```

3.2 — Gestión de ruido: auto-aprobados

Capa	Comportamiento	Ejemplo
Silencio (default)	Auto-aprobados no generan notificación individual	Los 252 auto-approves diários van en silencio
Batch en heartbeat	Cada 10 min, incluir línea: "🔒 12 auto-aprobados"	Dentro del screenshot del dashboard
Confirmación "Siempre"	Cuando el usuario presiona ✅ ✅, confirmar qué patrón se guardó	`Bash(./qa/scripts/qa-android.sh:*)` agregado
Comando /permisos	Bajo demanda: listar últimos N permisos con estado	`/permisos 10` en Telegram

4. Requisitos funcionales y no funcionales

4.1 — Requisitos MUST (obligatorios)

ID	Requisito	Criterio de aceptación	Estado
RF-01	Notificación garantizada de permisos	≥99% de permisos llegan a Telegram (vía approver o fallback) en <5s	⚠ Parcial
RF-02	Aprobación remota vía Telegram	Botón presionado → Claude reanuda en <2s	✅ OK
RF-03	Persistencia "Siempre"	Patrón guardado en settings.local.json, efectivo cross-session	✅ OK
RF-04	Fallback cuando hook falla	Notification(permission_prompt) notifica si no hay approver activo	⚠ Parcial
RF-05	Timeout y escalación (4 intentos)	Secuencia: 30+15+10+10 min con urgencia creciente	✅ OK
RNF-04	Sin dependencia de hooks inconsistentes	El sistema funciona aunque PermissionRequest no se dispare	❌ Gap

4.2 — Requisitos SHOULD (deseables)

ID	Requisito	Estado
RF-06	Resumen de permisos auto-aprobados en heartbeat	❌ No implementado
RF-07	Health-check detecta prompts huérfanos	⚠ Parcial
RNF-01	Latencia ≤3s (approver) / ≤5s (fallback)	✅ En condiciones normales
RNF-02	Fiabilidad ≥99% con mecanismo de medición	❌ Sin auditoría
RNF-03	Disponibilidad cuando Telegram está caído	⚠ Sin retry en sendMessage

4.3 — Requisitos COULD (opcionales)

ID	Requisito
RF-06.5	Comando `/permisos N` en Telegram
RF-07.6	Comando `/health` forzado en Telegram
UX-01	Botón "🔍 Ver más" para comandos truncados
ARCH-01	Endpoint HTTP en Commander para reemplazar polling de archivo

5. Gaps críticos a resolver

🔴 Gap 1: PID file se escribe DESPUÉS del sendMessage

Problema: El approver escribe `approver-active.pid` en línea 426, DESPUÉS de enviar el mensaje a Telegram. En la ventana de 1-3s entre el envío y la escritura del PID, la Notification puede dispararse, no encontrar PID, y enviar un duplicado.

Fix: Escribir PID file al INICIO de `processInput()`. Si es auto-approve, borrarlo al salir.

Impacto: Elimina race condition RF-01.4

Esfuerzo: 5 líneas de código

🔴 Gap 2: EPERM tratado como "proceso muerto"

Problema: `process.kill(pid, 0)` en Windows lanza `EPERM` cuando el proceso existe pero hay diferencia de permisos. El código actual lo trata como proceso muerto.

Fix: En todos los archivos que usan PID check (`notify-telegram.js`, `commander-launcher.js`, `health-check.js`), cambiar: `catch(e) { return e.code === "EPERM"; }`

Impacto: Elimina falsos negativos en Windows

Esfuerzo: 3 archivos, 1 línea cada uno

🔴 Gap 3: sendMessage sin retry

Problema: Si el primer `sendMessage` falla (error transitorio de red, rate limit), el approver hace `exit(0)` inmediatamente. Un error de 1-2 segundos causa fallback completo.

Fix: Agregar 1 retry con 3s de backoff antes del `exit(0)`.

Impacto: Resuelve FE-02 (Telegram API caída transitoria)

Esfuerzo: 10 líneas

🟡 Gap 4: Botón "Siempre" es texto libre, no botón

Problema: La opción "siempre" solo existe como instrucción de texto. El usuario mobile ve 2 botones y los usa. El concepto "siempre" queda escondido.

Fix: Agregar tercer botón: `{ text: "✅✅ Siempre", callback_data: "always:" + requestId }`

Impacto: UX mejora significativa — menos comandos repetidos

Esfuerzo: 3 líneas (el commander ya soporta `always:`)

🟡 Gap 5: Callback perdido cuando Commander estaba caído

Problema: Si el Commander cae por >30s (POLL_TIMEOUT_SEC), el callback de Telegram puede descartarse de la cola de getUpdates. El usuario cree haber aprobado pero Claude sigue esperando.

Fix: Al arrancar, el Commander verifica preguntas `pending` con `telegram_message_id` y solicita reenvío de respuesta.

Impacto: Resuelve FE-04 (gap crítico del PO)

Esfuerzo: ~30 líneas

🟡 Gap 6: answerCallbackQuery tiene timeout de ~15s

Problema: Si el Commander está ocupado procesando otro comando cuando llega un callback de permiso, puede perder la ventana de 15s para acknowledgearlo.

Fix: Priorizar callbacks de permisos: procesarlos inmediatamente antes de ejecutar comandos largos.

Impacto: Evita "loading..." infinito en Telegram al presionar botón

Esfuerzo: ~20 líneas

6. Plan de implementación

Fase 1 — Fixes críticos (mismo día) MUST

#	Cambio	Archivo	Líneas
1.1	Escribir PID file al inicio de processInput (antes de auto-approve check)	permission-approver.js	~8
1.2	Fix EPERM en process.kill(pid, 0) para Windows	notify-telegram.js, commander-launcher.js, health-check.js	~3
1.3	Retry 1x con 3s backoff en sendMessage del approver	permission-approver.js	~12
1.4	Agregar botón ✅ ✅ Siempre a inline keyboard	permission-approver.js	~3
1.5	Nuevo header visual por estado (PERMISO REQUERIDO / RECORDATORIO / etc.)	permission-approver.js	~25

Fase 2 — Robustez (1-2 días) SHOULD

#	Cambio	Archivo	Líneas
2.1	Commander: verificar pending questions al arrancar y solicitar reenvío	telegram-commander.js	~30
2.2	Commander: priorizar callbacks de permisos sobre comandos largos	telegram-commander.js	~20
2.3	Heartbeat: incluir conteo de auto-aprobados	heartbeat lógica (activity-logger o health-check)	~15
2.4	Confirmación visual al presionar "Siempre" (patrón guardado)	telegram-commander.js	~10
2.5	Health-check: detectar prompts huérfanos (pending >5 min sin approver vivo)	health-check.js	~20

Fase 3 — Mejoras opcionales (backlog) COULD

#	Cambio	Archivo
3.1	Endpoint HTTP en Commander para reemplazar polling de archivo	telegram-commander.js + permission-approver.js
3.2	Comando `/permisos N` para auditoría	telegram-commander.js
3.3	Botón "🔍 Ver más" para comandos truncados	permission-approver.js + telegram-commander.js
3.4	Script de auditoría automática (medir tasas de RF/RNF)	scripts/audit-permissions.js

7. Métricas de éxito

≥99%
Tasa de notificación (llegan a Telegram)

≥80%
Resolución remota (sin ir a consola)

<3s
Latencia P95 (permiso → Telegram)

<2s
Latencia P95 (botón → Claude)

0%
Reaprobación de patrones "Siempre"

≥99.5%
Disponibilidad del Commander

Cómo medir

Todas las métricas se calculan desde `hook-debug.log`. Cada permiso tiene un `requestId` único con timestamps de: invocación, envío, respuesta, resolución. Un script de auditoría puede calcular las tasas automáticamente.

Resumen ejecutivo: El sistema actual funciona en el ~70% de los casos. Los 6 gaps identificados tienen fixes de bajo esfuerzo (Fase 1: ~50 líneas, Fase 2: ~95 líneas). La Fase 1 sola debería llevar la fiabilidad a ≥95%. La Fase 2 cubre edge cases para alcanzar el ≥99%.

🤖 Generado con Claude Code — Intrale Platform 2026